

模块四: 数据工程 (上)

第十六讲: 模块收官项目——电影市场洞察报告

【解释】

项目式学习 (PBL)

本讲采用项目式学习模式。与前三讲的知识点教学不同, L16 不引入新技术, 而是要求学生综合运用已学技能完成一个端到端的分析任务。

项目的核心教学价值在于: 让学生体验"从原始数据到可交付结论"的完整链路, 建立解决复杂问题的工程自信。

影市场洞察报告

模块四收官项目

一、项目背景与任务简报

综合运用 L13-L15 全部技能

[解释]

第一章：项目背景

本章设定项目情境、明确任务目标、展示与前三讲的技能对应关系。

本章包含 3 页内容：项目背景与数据集介绍、项目路线图、技能回顾地图。

项目背景：TMDB 5000 电影数据集

你手里有一份 `movies.csv`，包含 TMDB (The Movie Database) 的 **4803 部** 电影数据。

核心字段

字段	说明
<code>title</code>	电影标题 (英文)
<code>genre</code>	主类型 (Action、Comedy 等)
<code>budget</code>	制片成本 (美元)
<code>revenue</code>	票房收入 (美元)
<code>vote_average</code>	用户评分 (0-10)
<code>overview</code>	英文剧情简介

数据现状

- 约 1000 部电影的 `budget` 缺失
- 约 1400 部电影的 `revenue` 缺失

你的任务

1. **EDA 体检**：检查缺失值和异常分布
2. **清洗计算**：处理缺失、计算 ROI
3. **类型分析**：哪个类型的平均回报率最高？
4. **LLM 加工**：为高分电影生成中文推荐语
5. **导出清单**：产出可交付的速查 CSV

项目成功标准：不是写了多少行代码，而是否走通了从原始数据到可交付结论的完整链路。

[解释]

TMDB 5000 数据集

TMDB (The Movie Database) 是一个社区维护的电影数据库，类似于 IMDb。TMDB 5000 是 Kaggle 上最常用的电影分析数据集之一。

原始数据的 `genres` 列是 JSON 格式 (如 `[{"id": 28, "name": "Action"}]`)，已在预处理阶段展开为可读字符串。`budget` 和 `revenue` 中的 0 值已替换为缺失值 (NaN)，因为“成本为 0”在业务上不合理。

项目路线图：5 步走完完全链路

流程与技能对照

Step	任务	对应技能
Step 1	读取与体检	L13 EDA
Step 2	清洗与 ROI 计算	L14 清洗
Step 3	类型统计分析	L13 分组统计
Step 4	LLM 中文推荐语	L15 LLM
Step 5	导出速查清单	L14 结构化输出

数据流向

```
movies.csv (4803 部)
↓ Step 1: 读取 + 体检
↓ Step 2: 清洗 (→ 3229 部有效)
↓ Step 3: groupby 统计 ROI
↓ Step 4: LLM 处理 overview (筛选后约 175 部)
↓ Step 5: 导出 must_watch.csv
```

💡 和 L15 的论文助手一样：**先打样再量产**——Step 4 只对筛选后的高分电影调用 LLM，而非全部 5000 部。

[解释]

与 L13-L15 的技能对应

L16 的设计逻辑是"不引入新知识，只综合运用已学技能"。每个 Step 都能在前三讲中找到对应的原型：

- Step 1 对应 L13 的 `df.info()` + 缺失值检查
- Step 2 对应 L14 的 `dropna()` + 特征工程
- Step 3 对应 L13 的 `groupby().agg()`
- Step 4 对应 L15 的 OpenAI SDK + Prompt 设计 + JSON 解析
- Step 5 对应 L14 的 `to_csv()` 结构化导出

二、Step 1-2：读取与清洗

复习 L13 EDA 与 L14 数据清洗

[解释]

第二章：读取与清洗

本章复习 L13 的 EDA 体检和 L14 的数据清洗技能。包含 3 页内容：Step 1 读取与体检、Step 2 清洗与 ROI 计算、审计验证。

Step 1: 读取与体检

任务

1. 读取 `movies.csv`
2. 查看前 5 行
3. 用 `df.info()` 检查缺失情况

代码

```
import pandas as pd

df = pd.read_csv('movies.csv')
print(df.shape)
print(df.info())
```

思考

- `budget` 和 `revenue` 各有多少缺失?
- 哪些列的数据类型需要关注?

预期输出

```
(4803, 10)
title          4803 non-null
genre          4778 non-null
budget         3766 non-null ← 有 1037 部缺失
revenue        3376 non-null ← 有 1427 部缺失
vote_average   4803 non-null
overview       4800 non-null
```

[解释]

数据完整性检查

`df.info()` 是数据体检的标准工具。它能快速告诉我们三件事：数据量（4803 行）、每列的非空值数量、数据类型。

`budget` 和 `revenue` 的缺失值在预处理阶段已从 "0 值" 转换为 `NaN`。原始数据中的 0 值在业务上不合理（没有电影的制片成本是 0），因此将其标记为缺失是正确的数据清洗决策。

Step 2: 清洗与 ROI 计算

任务

1. 删除 `budget` 或 `revenue` 缺失的行
2. 计算 ROI (投资回报率)
3. 打印前 5 行验证

代码

```
# 删除缺失值
df = df.dropna(subset=['budget', 'revenue'])
print(f"清洗后: {len(df)} 部电影")

# 计算 ROI
df['roi'] = (df['revenue'] - df['budget']) / df['budget']

# 验证
print(df[['title', 'budget', 'revenue', 'roi']].head())
```

ROI 公式

$ROI = (\text{票房} - \text{成本}) / \text{成本}$

ROI 值	含义
ROI > 2	高回报 (赚了 2 倍以上)
ROI = 0	保本
ROI < 0	亏损

预期输出

```
清洗后: 3229 部电影
   title  budget  revenue  roi
0  Avatar  237000000.0  2787965087  10.76
1  Pirates of the Caribb.  300000000.0  961000000  2.20
```

[解释]

特征工程与 ROI

ROI (Return on Investment) 是商业分析的核心指标。原始数据只有 `budget` 和 `revenue`，无法直接比较盈利能力——因为花 2 亿赚 5 亿和花 100 万赚 500 万，虽然绝对利润差距巨大，但 ROI 都是 1.5。

`dropna(subset=['budget', 'revenue'])` 是 Pandas 的条件删除操作：只删除指定列为 NaN 的行，保留其他列有缺失但 `budget/revenue` 完整的记录。

审计验证：ROI 是否合理？

小样本验证（L13 .head(5) 原则）

```
# 检查 ROI 的分布
print(f"ROI 最小值: {df['roi'].min():.2f}")
print(f"ROI 最大值: {df['roi'].max():.2f}")
print(f"ROI 中位数: {df['roi'].median():.2f}")
print(f"亏损电影: {(df['roi'] < 0).sum()} 部")
```

手动验算

随机挑一部电影，用计算器验证：

```
# Avatar: budget=237M, revenue=2788M
roi = (2788 - 237) / 237 # = 10.76 ✓
```

预期合理范围

- 大部分电影 ROI 在 -1 到 10 之间
- 中位数应略大于 0（多数电影微利或微亏）
- 如果 ROI 最大值超过 1000，需要排查异常数据

🎯 L13 的核心习惯：先用 `.head(5)` 打样，再跑全量。如果 ROI 出现负无穷大或 NaN，说明代码逻辑有问题——budget 可能有 0 值没被清除。

[解释]

审计验证的工程价值

在数据分析中，“代码跑通了”不等于“结果是对的”。审计验证分两步：

1. **统计审计**：检查结果的分布是否符合业务常识（ROI 中位数应略大于 0）
2. **抽样验证**：随机挑 1-2 条记录，用计算器手算，和程序输出对照
如果发现 ROI 出现 `inf`（无穷大），说明有 `budget=0` 的记录没被清除（除以 0）。如果出现 NaN，说明有非数值类型混入了计算。

三、Step 3：统计分析

复习 L13 分组统计

[解释]

第三章：统计分析

本章复习 L13 的分组统计技能。包含 2 页内容：按类型统计平均 ROI、洞察解读。

Step 3: 类型对比——哪个类型的电影最值得投资？

任务

1. 按 `genre`（主类型）分组
2. 计算平均 ROI
3. 按均值降序排列

思考

- 票房冠军（Action/Adventure）的 ROI 一定最高吗？
- 小成本类型会不会有更高的回报率？

代码

```
# 按类型分组，计算平均 ROI
roi_by_genre = (
    df.groupby('genre')['roi']
      .agg(['mean', 'count'])
      .sort_values('mean', ascending=False)
)

print(roi_by_genre.head(10))
```

预期输出

	mean	count
genre		
Horror	11.95	192
Animation	6.21	99
Adventure	4.72	287
Drama	4.17	738
Comedy	4.09	632
...		

 **发现：**Horror（恐怖片）的平均 ROI 排名前列——因为它的制片成本低，即使票房不高，回报率也很可观。这和 L13 中“用数据打破直觉”的逻辑完全一致。

[解释]

分组统计与辛普森悖论

`groupby('genre')['roi'].agg(['mean', 'count'])` 同时计算了平均值和样本量。加上 `count` 列很重要——如果某个类型只有 3 部电影，其中 1 部偶然爆火，平均 ROI 就会被异常拉高，不具备统计参考价值。在解读分组统计结果时，需要同时关注平均值和样本量。样本量太小的类型（`count < 10`），其统计结论应谨慎对待。

洞察解读：数据驱动的投资建议

从数据到决策

基于 ROI 统计，可以形成两种投资策略：

策略 A：高回报型

- 关注 Horror、Animation 等高 ROI 类型
- 特点：低成本、高回报率，但票房天花板低
- 风险：个别低成本电影可能亏损严重

策略 B：稳健型

- 关注 Action、Adventure 等票房大盘类型
- 特点：成本高，但票房基数大，ROI 趋于稳定
- 风险：单片亏损金额大

ROI 分析的局限性

⚠ 注意：平均 ROI 只是一个维度。完整的投资分析还需要考虑：

- 票房的方差（稳定性）
- 类型的市场容量（总票房规模）
- 时间趋势（某类型是否在上升/衰退）

💡 本讲的分析是入门级的单维度统计。更完整的多维分析方法，将在后续课程中逐步引入。

[解释]

数据分析的边界

将 ROI 统计转化为投资策略，体现了"数据驱动决策"的基本思路。但需要注意分析的局限性：

1. **平均值的陷阱**：平均 ROI 高不代表每部都赚钱。如果方差很大，风险也大。
2. **样本偏差**：TMDB 数据集主要覆盖欧美市场，中国市场的情况可能不同。
3. **因果 vs 相关**：ROI 高的类型不一定"因为类型好所以赚钱"，可能是因为这些类型吸引了更优秀的制片团队。

四、Step 4-5：LLM 加工与报告导出

复习 L15 的 LLM API 调用

[解释]

第四章：LLM 加工与导出

本章复习 L15 的 LLM API 调用技能，并完成项目交付。包含 4 页内容：筛选高分电影、Prompt 设计、批量处理、导出清单。

Step 4: 筛选 + LLM 生成中文推荐语

筛选条件

先从 3200+ 部电影中筛选出值得推荐的：

- 评分 > 7.5 (口碑好)
- ROI > 1 (至少赚了一倍)

```
# 筛选高分高回报电影
candidates = df[
    (df['vote_average'] > 7.5) &
    (df['roi'] > 1)
].copy()

print(f"筛选出 {len(candidates)} 部电影")
```

为什么先筛选？

- 减少 LLM 调用次数 → 节省成本
- 只对有价值的电影生成推荐语
- 和 L15 的"先分类再摘要"逻辑一致

Prompt 设计 (复习 L15)

```
RECOMMEND_PROMPT = """你是电影推荐助手。
```

```
电影标题: {title}
英文简介: {overview}
```

```
## 规则
1. 生成一句中文推荐语 (20-30字)
2. 突出电影的核心卖点
3. 语言简洁有力
4. 同时翻译标题
```

```
返回 JSON (不要包裹在代码块中):
{"title_zh": "中文标题",
 "one_liner": "一句话推荐语"}"""
```

💡 Prompt 结构和 L15 的摘要
Prompt 完全一致：角色 + 输入数据 + 规则 + 输出格式。

[解释]

Prompt 设计的复用

L15 的分类 Prompt 和摘要 Prompt 是两个不同的模板，但结构完全相同：角色设定 + 数据输入 + 规则约束 + 输出格式。L16 的推荐语 Prompt 沿用了这个结构，只是任务从"生成论文摘要"变成了"生成电影推荐语"。这种 Prompt 结构的可复用性，是 Prompt 工程的核心价值：掌握了模板，就能快速适配不同的任务场景。

批量处理与成本控制

代码（复习 L15 批量流水线）

```
from openai import OpenAI
import json, time

client = OpenAI(
    api_key=api_key,
    base_url="https://api.siliconflow.cn/v1"
)
MODEL = "deepseek-ai/DeepSeek-V3.2"

results = []
for i, row in candidates.iterrows():
    try:
        resp = client.chat.completions.create(
            model=MODEL,
            messages=[{
                "role": "user",
                "content": RECOMMEND_PROMPT.format(
                    title=row['title'],
                    overview=row['overview']
                )
            }],
            temperature=0.3,
            max_tokens=200
        )
        data = json.loads(
            resp.choices[0].message.content
        )
        results.append({
            'title': row['title'],
            'title_zh': data.get('title_zh', ''),
            'one_liner': data.get('one_liner', ''),
        })
        time.sleep(0.5)
    except Exception as e:
        print(f"失败: {row['title']}: {e}")

print(f"生成 {len(results)} 条推荐语")
```

工程要点（与 L15 完全一致）

1. try/except 错误处理

单部失败不影响整体流水线

2. time.sleep(0.5) 频率控制

避免 API 限流

3. 先筛选再调用

从 3200 部缩减到约 175 部，节省 95% 的 API 成本

成本估算

- 175 部电影 × 约 300 Token/部
- 总计约 52K Token
- 费用 < ¥0.15

⚠ L15 的核心守则依然有效：API Key 通过 `getpass()` 安全输入，LLM 输出需要抽检。

[解释]

批量处理的工程模式

L16 的批量处理代码是 L15 的直接复用，核心结构完全一致：`for` 循环 + `try/except` + `time.sleep()`。唯一的变化是 Prompt 的内容——从“论文摘要”变成了“电影推荐语”。

这种代码复用性说明了一个工程事实：LLM 调用的代码框架是通用的，不同任务之间的差异主要在 Prompt 设计，而非代码结构。

Step 5: 导出速查清单

合并结果

```
# 将 LLM 结果合并回主表
rec_df = pd.DataFrame(results)
final = candidates.merge(
    rec_df, on='title', how='left'
)

# 选择关键列并排序
output = final[[
    'title', 'title_zh', 'genre',
    'vote_average', 'roi', 'one_liner'
]].sort_values('roi', ascending=False)

# 导出
output.to_csv('must_watch.csv', index=False)
print(f"✅ 已导出: {len(output)} 部电影")
```

输出示例

中文标题	类型	评分	ROI	推荐语
阿甘正传	Comedy	8.2	11.33	一个简单灵魂的传奇人生...
盗梦空间	Action	8.1	4.16	多层梦境中的终极夺心...
星际穿越	Adventure	8.1	3.09	穿越虫洞的父女羁绊...

💡 最终产出就是这张表——中文标题、类型、评分、ROI、一句话推荐语。一份可以直接交给非技术人员的速查清单。

[解释]

数据产品的交付

数据分析的终点不是代码，而是可交付的成果。CSV 文件是最通用的交付格式——Excel、Google Sheets、甚至微信都能直接打开。

`merge(rec_df, on='title', how='left')` 是 Pandas 的表关联操作：以电影标题为键，将 LLM 结果合并到主表。`how='left'` 表示保留左表（candidates）的所有行，即使某些电影的 LLM 调用失败了。

五、总结与反思

模块四能力闭环

【解释】

第五章：总结与反思

本章回顾项目全流程，总结常见问题，并展示模块四（L13-L16）的完整能力闭环。
本章包含 3 页内容：避坑指南、模块总结、课后练习预告。

避坑指南

常见报错

1. KeyError: 'budget'

列名拼错了。先打印 `df.columns` 再确认

2. ZeroDivisionError

budget 中有 0 值未被清除。清洗时要用 `replace(0, pd.NA)` 再 `dropna()`

3. JSONDecodeError

LLM 返回了 Markdown 代码块包裹的 JSON。用 L15 学过的 `re.search` 提取

逻辑陷阱

4. 加工顺序反了

必须先 `dropna()` 再计算 ROI，而不是反过来。这是 L13 教的"先过滤再计算"原则

5. 对全量数据调 LLM

忘了先筛选，对 3200 部电影全部调用 API → 浪费时间和费用

6. 不抽检 LLM 输出

LLM 可能为恐怖片生成了"温馨治愈"的推荐语——必须抽查

[解释]

技术报错 vs 逻辑错误

技术报错 (KeyError、ZeroDivisionError) 是"显性错误"——程序会停止运行并提示错误位置。逻辑错误 (加工顺序反了、对全量调 LLM) 是"隐性错误"——程序正常运行但结果不正确。

L13 教的"先过滤再计算"和"先打样再量产"两条原则，正是为了防御隐性错误。在实际工作中，隐性错误造成的损失远大于显性错误。

模块四总结：L13-L16 能力闭环

能力清单

- **✓ L13**: EDA 体检、分组统计、数据驱动洞察
- **✓ L14**: 爬虫采集、多源对齐、SSOT 原则
- **✓ L15**: LLM API 调用、Prompt 工程、JSON 解析
- **✓ L16**: 端到端项目、综合调度、成果交付

下一阶段预告

模块五：数据可视化

我们已经学会了分析和产出结论。下一步要学习如何通过图表把这些结论直观地展示出来：

- 散点图：ROI vs 评分的关系
- 柱状图：各类型的平均 ROI 对比
- 交互图表：让用户自己筛选和探索

核心守则

- 先打样再量产（L13 → L16 贯穿始终）
- 先过滤再计算（EDA → ROI → LLM 层层筛选）
- API Key 安全（L15 核心守则）
- 抽检 LLM 输出（不盲信 AI）

[解释]

模块四的知识体系

L13-L16 构成了数据工程的完整技能栈：

- **L13**: 结构化数据处理（EDA、清洗、统计分析）
- **L14**: 数据采集（HTML 爬虫、多源对齐、SSOT 原则）
- **L15**: 非结构化文本处理（RSS API、LLM 分类与摘要、JSON 解析）
- **L16**: 综合项目（端到端链路验证、成果交付）

模块五将在这个基础上增加"可视化表达"能力——把数据分析的结论转化为直观的图表，是数据分析师的核心竞争力之一。

课后练习

以下三个挑战基于同一份 `movies.csv` 数据集，难度递进。

挑战一（基础）：谁是票房之王？

按导演分组（需要从 `tmdb_5000_credits.csv` 中提取导演），统计每位导演的总票房，找出 Top 10。

挑战二（进阶）：电影越来越贵了吗？

从 `release_date` 中提取年份，按年代统计平均 budget，观察制片成本的时间趋势。

挑战三（挑战）：寻找"遗珠"

筛选出 `budget < 1000` 万美元 但 `vote_average > 7.5` 的电影。这些是典型的"低成本高口碑"作品。用 LLM 为它们生成中文推荐语。

[解释]

课后练习的分层设计

三个挑战覆盖了不同的能力层次：

- **基础**： `groupby` + `sum` + 多表合并 (L13 + L14)
- **进阶**：日期处理 + 趋势分析 (L13 + 新技能引导)
- **挑战**：多条件筛选 + LLM 调用 (L14 + L15)

特别注意挑战题中 "`budget < 1000` 万" 的条件——数据中的 `budget` 单位是美元，1000 万 = 10,000,000。这个单位换算是实际分析中的常见陷阱。